

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА
Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине **«Программное обеспечение мехатронных и робототехнических систем»**

Тема работы: «Отладка программного обеспечения робототехнических систем с использованием виртуального моделирования»

Студенты группы: КРБО-02-23 Евлампиев И. С. _____
Мальчуковский М. О. _____
Проклин В. С. _____
Филатов Г. А. _____
Преподаватель: Морозов А. А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

1 Цель работы

Получение навыков моделирования объекта управления в промышленных системах автоматического управления и создания функциональных блоков в среде B&R Automation Studio версии 4.5.2 (без функционального оборудования, Hardware-in-the-loop (HITL)).

2 Задание

Создать виртуальную систему управления, включающую модель объекта управления (двигателя постоянного тока), ПИ-регулятор, сумматор и обратную связь. Передаточная функция объекта управления:

$$W = \frac{1}{k_e \cdot (T_м s + 1)}$$

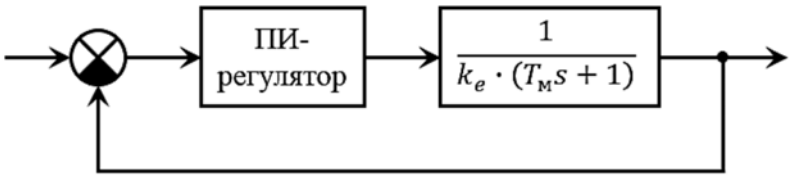


Рис. 1.1. Структура системы управления.

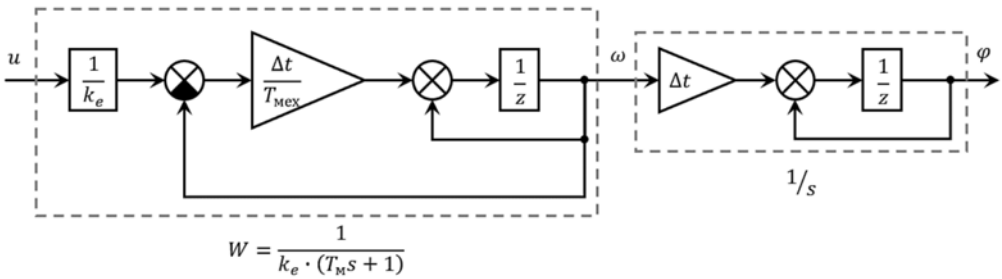


Рис. 1.2. Структура объекта управления.

Реализовать модель в виде функциональных блоков FB_Motor (модель двигателя), FB_Regulator (ПИ-регулятор со схемой противонакопления) и FB_Integrator (интегрирующее звено), объединить их в основной программе, подать на систему ступенчатое воздействие и сравнить переходной процесс двигателя без регулятора и с регулятором.

3 Основная часть

3.1 Описание действий и использованного аппарата

Работа выполнена в среде разработки B&R Automation Studio с использованием симуляции контроллера (Automation Runtime Simulation), без подключения физического стенда. Язык программирования функциональных блоков — ANSI C.

В рамках проекта были созданы следующие объекты: «ANSI C Program» и библиотека «ANSI C Library» MotorControl с тремя функциональными блоками: FB_integrator (интегрирующее звено), FB_Motor (модель двигателя постоянного тока) и FB_Regulator (цифровой ПИ-регулятор со схемой противонакопления). Коэффициенты регулятора рассчитаны методом желаемой передаточной функции (решение обратной задачи динамики при Тж = 0.02: Кп = ke·Тм/Тж, Ки = ke/Тж).

Моделирование системы управления скоростью вращения двигателя постоянного тока выполнено путём численного (дискретного) интегрирования разностных уравнений с шагом расчёта dt = 0,01 с. Результаты сняты средством Trace, по графикам высчитаны численные значения.

3.2 Схема модели объекта управления и регулятора

Общая структура системы управления, включающая ПИ-регулятор и модель объекта управления (двигателя) с обратной связью по скорости, приведена на рисунке 1.

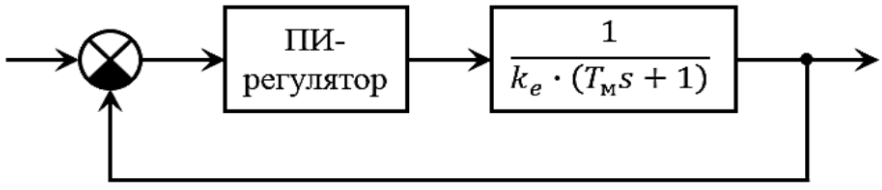


Рисунок 1 – Структура системы управления (ПИ-регулятор и объект управления с обратной связью)

Внутренняя структура модели объекта управления (двигателя постоянного тока), реализующая передаточную функцию W = 1/(ke·(Тм·s+1))

в дискретном виде с получением скорости ω и положения φ, приведена на рисунке 2.

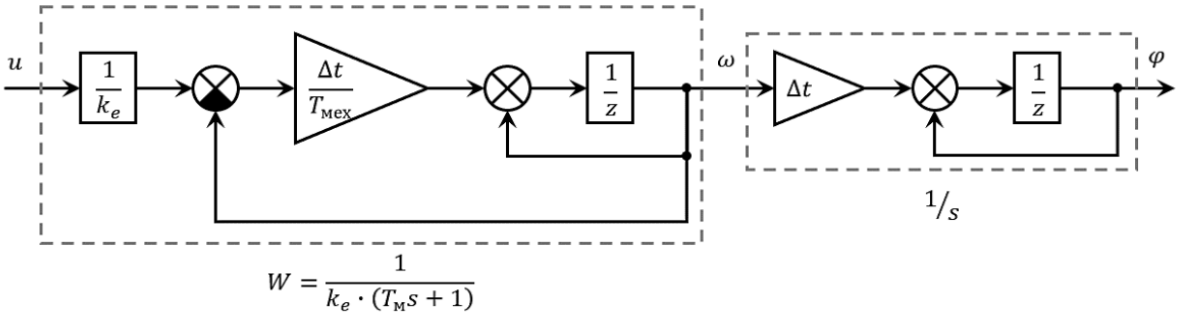


Рисунок 2 – Структура объекта управления (модель двигателя постоянного тока)

Структура ПИ-регулятора, включающая пропорциональную и интегральную составляющие, блоки ограничения напряжения и схему противонакопления (anti-windup), предотвращающую избыточное накопление интеграла при насыщении выхода, приведена на рисунке 3.

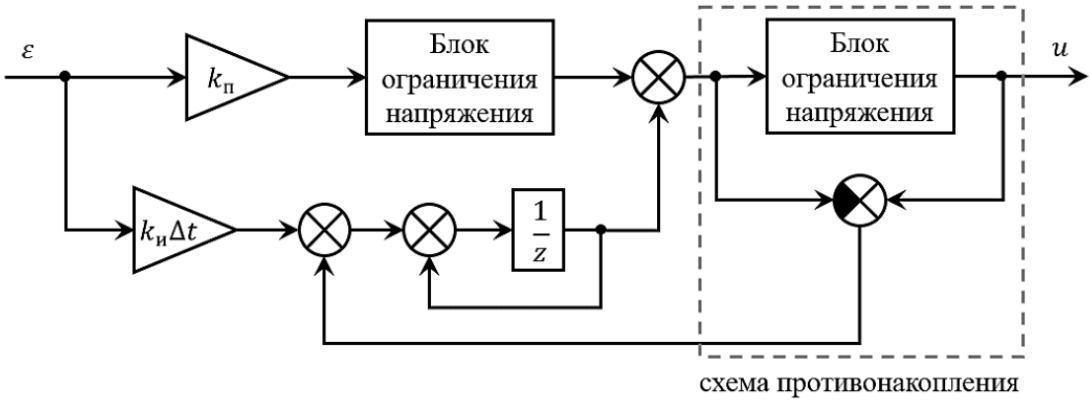


Рисунок 3 – Структура ПИ-регулятора со схемой противонакопления

3.3 Описание функциональных блоков

FB_Integrator — модель интегрирующего звена. Накапливает сумму произведений входного значения на шаг расчёта, реализуя передаточную функцию 1/z (дискретный интегратор).

Таблица 1 – Конфигурация функционального блока FB_Integrator

Конфигурация	Имя	Тип данных	Описание
вход	in	REAL	вход интегрирующего звена
выход	out	REAL	выход интегрирующего звена
внутр. состояние	dt	REAL	шаг расчёта, с

FB_Motor — модель двигателя постоянного тока на основе передаточной функции W = 1/[ke·(Тм·s+1)]. Использует внутренний экземпляр FB_Integrator для вычисления скорости и положения вала.

Таблица 2 – Конфигурация функционального блока FB_Motor

Конфигурация	Имя	Тип данных	Описание
вход	u	REAL	входное напряжение, В
выход	w	REAL	частота вращения, об/мин
выход	phi	REAL	положение вала, рад
внутр. состояние	integrator	FB_Integrator	интегратор скорости
внутр. состояние	Tm	REAL	электромеханическая постоянная времени, с
внутр. состояние	ke	REAL	постоянная ЭДС двигателя, В·мин/об
внутр. состояние	dt	REAL	шаг расчёта, с

FB_Regulator — модель ПИ-регулятора со схемой противонакопления интегральной составляющей и ограничением выходного напряжения.

Таблица 3 – Конфигурация функционального блока FB_Regulator

Конфигурация	**Имя**	**Тип данных**	**Описание**
вход	e	REAL	рассогласование между уставкой и скоростью, об/мин
выход	u	REAL	напряжение на входе ДПТ, В
внутр. состояние	k_p	REAL	пропорциональный коэффициент
внутр. состояние	k_i	REAL	интегральный коэффициент
внутр. состояние	integrator	FB_Integrator	интегратор регулятора
внутр. состояние	max_abs_value	REAL	граница ограничения выхода, В
внутр. состояние	dt	REAL	шаг расчёта, с

3.4 Описание структуры программного обеспечения

Программное обеспечение состоит из подпрограммы инициализации Init.c (задаёт коэффициенты регулятора, параметры двигателей и шаг расчёта) и циклической подпрограммы Cyclic.c, выполняемой каждые 2 мс. В Cyclic.c реализованы: формирование ступенчатого воздействия с помощью счётчика counter, расчёт рассогласования, вызов FB_Regulator для расчёта управляющего напряжения, вызов FB_Motor для двигателя, охваченного регулятором, и параллельный расчёт второго двигателя (fb_motor_2),

работающего без регулятора (в разомкнутом контуре), для сравнения переходных процессов.

Коэффициенты ПИ-регулятора рассчитаны методом обратной задачи динамики (при $T_ж = 1$ с): $K_п = k_е \cdot T_м / T_ж$, $K_и = k_е / T_ж$. В соответствии с расчётом в инициализации приняты значения $k_p = 0{,}24$ и $k_i = 2{,}4$ при $k_е = 0{,}5$ и $T_м = 0{,}1$ с.

3.5 Результаты экспериментальных исследований

Для снятия переходных характеристик использовалось средство Trace, в конфигурацию которого добавлены переменные: speed (входное ступенчатое воздействие), fb_motor.w (скорость вращения двигателя с регулятором), fb_motor_2.w (скорость вращения двигателя без регулятора).

По результатам эксперимента видно, что двигатель с ПИ-регулятором отрабатывает ступенчатое задающее воздействие без статической ошибки и с меньшим временем переходного процесса по сравнению с двигателем, работающим в разомкнутом контуре (подача напряжения пропорционально уставке без обратной связи)(рисунок 4).

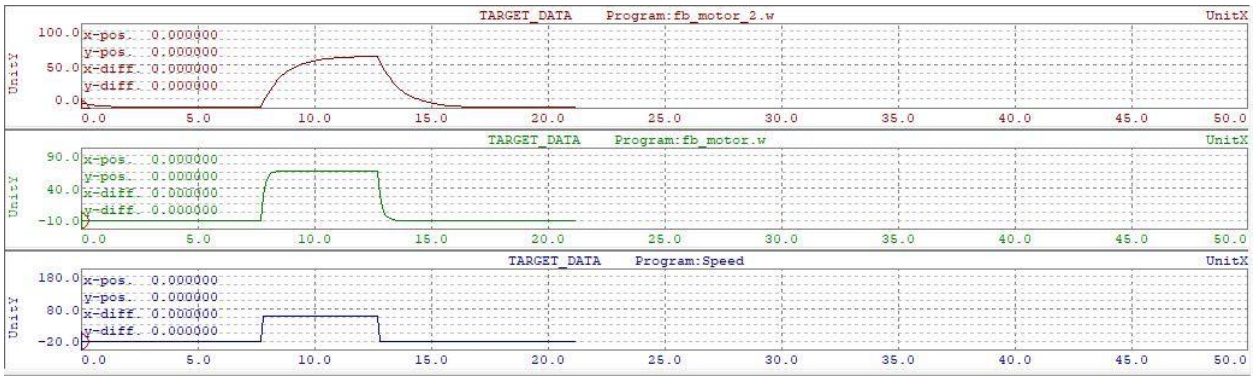


Рисунок 4 – Графики задающего воздействия speed, скорости двигателя без регулятора (fb_motor_2.w) и с ПИ-регулятором (fb_motor.w) на одном цикле ступенчатого воздействия

Курсором Trace в установившемся режиме (рисунок 5) считаны значения.

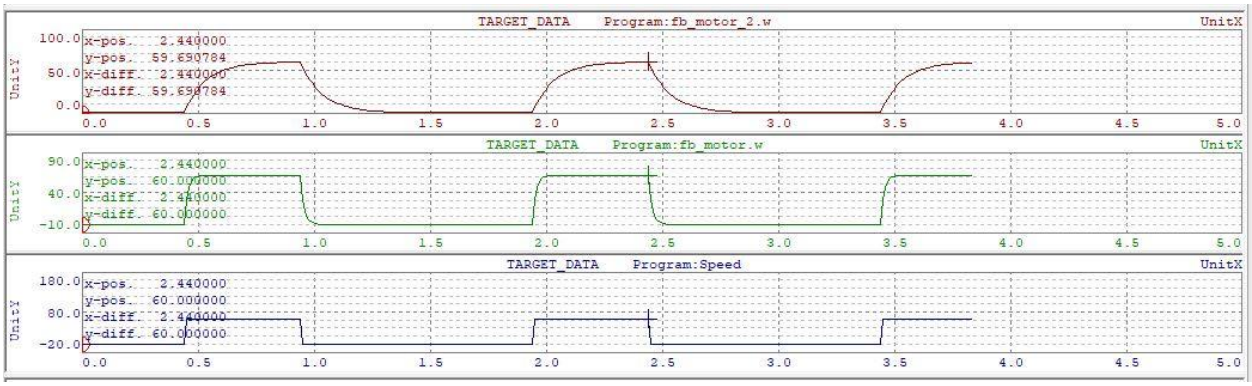


Рисунок 5 – Считывание значений сигналов курсором Trace

Оба двигателя выходят на установившийся режим быстро и без колебаний, но двигатель с ПИ-регулятором держит заданную скорость точнее, чем двигатель без регулятора и имеет меньшее время регулирования.

Name	Type	Reference	Scope	Constant	Retain	Replicable	Redundancy	Value	Description [1]
FB_Motor		☐				☑	Unusable		TODO: Add your comm
u	REAL	☐	VAR_INPUT	☐	☐	☐			[В]
u2	REAL	☐	VAR_INPUT	☐	☐	☑			
w	REAL	☐	VAR_OUTPUT	☐	☐	☐			[об/мин]
phi	REAL	☐	VAR_OUTPUT	☐	☐	☐			[рад]
w2	REAL	☐	VAR_OUTPUT	☐	☐	☑			
phi2	REAL	☐	VAR_OUTPUT	☐	☐	☑			
integrator	FB_Integrator	☐	VAR	☐	☐	☐			
Tm	REAL	☐	VAR	☐	☐	☐			
ke	REAL	☐	VAR	☐	☐	☐			
integrator2	FB_Integrator	☐	VAR	☐	☐	☑			
Tm2	REAL	☐	VAR	☐	☐	☑			
ke2	REAL	☐	VAR	☐	☐	☑			
dt	REAL	☐	VAR	☐	☐	☑			
FB_Regulator		☐				☐	Unusable		TODO: Add your comm
e	REAL	☐	VAR_INPUT	☐	☐	☐			
u	REAL	☐	VAR_OUTPUT	☐	☐	☐			
k_p	REAL	☐	VAR	☐	☐	☐			
k_i	REAL	☐	VAR	☐	☐	☐			
integrator	FB_Integrator	☐	VAR	☐	☐	☐			
iyOld	REAL	☐	VAR	☐	☐	☐			
max_abs_value	REAL	☐	VAR	☐	☐	☐			
dt	REAL	☐	VAR	☐	☐	☐			
FB_Integrator		☐				☑	Unusable		TODO: Add your comm
in	REAL	☐	VAR_INPUT	☐	☐	☐			
in2	REAL	☐	VAR_INPUT	☐	☐	☑			
out	REAL	☐	VAR_OUTPUT	☐	☐	☐			
out2	REAL	☐	VAR_OUTPUT	☐	☐	☑			
dt	REAL	☐	VAR	☐	☐	☐			
dt2	REAL	☐	VAR	☐	☐	☑			

Рисунок 6 – Переменные всех функциональных блоков

4 Выводы

В ходе выполнения лабораторной работы были получены навыки моделирования объекта управления (двигателя постоянного тока) и ПИ-регулятора в виде функциональных блоков на языке ANSI C в среде B&R Automation Studio. Реализована дискретная модель интегрирующего звена, на основе которой построены модели двигателя и регулятора со схемой противонакопления. Экспериментально подтверждено, что применение ПИ-регулятора, коэффициенты которого рассчитаны методом обратной задачи динамики, позволяет получить требуемое качество переходного процесса (отсутствие статической ошибки, приемлемое быстродействие) по сравнению с разомкнутой системой управления.

ПРИЛОЖЕНИЕ А

Листинг программы Init.c

```
#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
#endif
#define DT  0.01
#define T_j  0.02
#define T_m  0.1
#define K_e  0.048
#define UMAX  24

void _INIT ProgramInit(void)
{
    fb_motor.Tm = T_m;
    fb_motor.ke = K_e;
    fb_motor.dt = DT;
    fb_motor.integrator.dt = DT;
    fb_motor.w = 0;
    fb_motor.phi = 0;

    fb_motor_2.Tm = T_m;
    fb_motor_2.ke = K_e;
    fb_motor_2.dt = DT;
    fb_motor_2.integrator.dt = DT;
    fb_motor_2.w = 0;
    fb_motor_2.phi = 0;

    fb_controller.k_p = K_e*T_m/T_j;
    fb_controller.k_i = K_e/T_j;
    fb_controller.max_abs_value = UMAX;
    fb_controller.dt = DT;
    fb_controller.integrator.dt = DT;
    fb_controller.iyOld = 0;



    Speed = 0;
    Enable = 1;
    counter = 0;
}

#include <bur/plctypes.h>

#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
#endif
#define STEP  50
#define SPEED_SET  60
void _CYCLIC ProgramCyclic(void)
{
    counter++;
    if (counter < STEP)
        Speed = 0;
    else if (counter < 2 * STEP)
        Speed = SPEED_SET;
    else if (counter < 3 * STEP)
        Speed = 0;
    else
```

```
        counter = 0;
    if (Enable)
    {
        fb_controller.e = Speed - fb_motor.w;
        FB_Regulator(&fb_controller);
        fb_motor.u = fb_controller.u;
    }
    else
    {
        fb_controller.e = 0;
        fb_controller.integrator.out = 0;
        fb_controller.u = 0;
        fb_motor.u = 0;
    }

    FB_Motor(&fb_motor);
    fb_motor_2.u =Speed * fb_motor_2.ke;
    FB_Motor(&fb_motor_2);


}

#include <bur/plctypes.h>

#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
#endif

void _EXIT ProgramExit(void)
{

}
```

ПРИЛОЖЕНИЕ Б

Листинг функционального блока FB_Motor.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
    #include "MotorContr.h"
#ifdef __cplusplus
    };
#endif
/* TODO: Add your comment here */
void FB_Motor(struct FB_Motor* inst)
{
    inst->integrator.dt = inst->dt;
    inst->integrator.in = (inst->u / inst->ke - inst->w) / inst->Tm;
    FB_Integrator(&inst->integrator);
    inst->w = inst->integrator.out;

    inst->phi += inst->w * inst->dt;


    /*TODO: Add your code here*/
}
```

ПРИЛОЖЕНИЕ В

Листинг функционального блока FB_Regulator.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
    #include "MotorContr.h"
#ifdef __cplusplus
    };
#endif
/* TODO: Add your comment here */
void FB_Regulator(struct FB_Regulator* inst)
{
    REAL u_p = inst->e * inst->k_p;
    if (u_p > inst->max_abs_value) u_p = inst->max_abs_value;
    else if (u_p < -inst->max_abs_value) u_p = -inst->max_abs_value;
    REAL u_i = inst->integrator.out;
    REAL u_pre = u_p + u_i;

    inst->u = u_pre;
    if (inst->u > inst->max_abs_value) inst->u = inst->max_abs_value;
    else if (inst->u < -inst->max_abs_value) inst->u = -inst->max_abs_value;

    inst->integrator.dt = inst->dt;
    inst->integrator.in = (inst->e * inst->k_i) - (u_pre - inst->u);
    FB_Integrator(&inst->integrator);

    /*TODO: Add your code here*/
}
```

ПРИЛОЖЕНИЕ Г

Листинг функционального блока FB_integrator.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
    #include "MotorContr.h"
#ifdef __cplusplus
    };
#endif
/* TODO: Add your comment here */
void FB_Integrator(struct FB_Integrator* inst)
{
    inst->out += inst->in * inst->dt;
    /*TODO: Add your code here*/
}
```